NEXVIA-AEGIS: เอกสารข้อกำหนดข้อมูลของหุ่นยนต์

Robot Telemetry, Perception & Multi-Agent Data Specification

โครงการวิจัยและพัฒนาสถาปัตยกรรมความปลอดภัยอากาศยานไร้คนขับ 6-DOF และฝูงบิน

เวอร์ชัน 1.0 (พฤศจิกายน 2026) — สำหรับทีมวิศวกรรมระบบและปัญญาประดิษฐ์

1 บทนำและสถาปัตยกรรมการไหลของข้อมูล (System Data Flow)

ในระบบอากาศยานไร้คนขับ 6-DOF Quadrotor และระบบฝูงบิน Multi-Agent ของ **NEXVIA-AEGIS** ข้อมูลทุกตัวแปรถูกออกแบบขึ้นตามหลักฟิสิกส์การเคลื่อนที่และทฤษฎีความปลอดภัยทางคณิตศาสตร์อย่างเคร่งครัด ผังการไหลของข้อมูลหลักในระบบสรุปได้ดังนี้:

แผนผังขั้นตอนการประมวลผลข้อมูล (Data Pipeline)

1. **Sensor Suite (IMU, RTK-GPS, Depth Camera, LiDAR):** บันทึกสัญญาณดิบระดับความถี่สูง (100–500 Hz)
2. **State Estimation & Perception (EKF / UKF / Point Cloud):**
   - *Robot Kinematics:* พิกัด **p**, ความเร็ว **v**, การวางตัว $R \in SO(3)$, ความเร็วเชิงมุม $\boldsymbol{\omega}$
   - *Sensor Integrity:* ความแปรปรวน σ_p, σ_v, ค่าความมั่งคั่งมาร์ติงเกล M_t (Ville e-Process)
   - *Obstacle Geometry:* พิกัด $\mathbf{c}_{\text{obs}}$, รัศมี r_{obs}, ความเร็ว $\mathbf{v}_{\text{obs}}$ และ 3D ESDF / Safe Corridor Polytopes
3. **Byzantine Swarm Consensus (BRMC):** แลกเปลี่ยนแพ็กเกจข้อมูลระหว่างโดรนเพื่อนในฝูง พร้อมคัดกรองโหนดอันธพาล
4. **Safe RL Policy (PPO Actor-Critic):** รับเวกเตอร์สังเกตการณ์ 20 มิติ คำนวณแรงขับตามเป้าหมายภารกิจ
5. **Real-Time C++20 RTI-SQP NMPC-HOCBF Core:** คำนวณเกราะความปลอดภัยและแรงขับที่ปลอดภัย $\mathbf{F}^*_{\text{safe}}$ ภายในเวลา < 0.1 ms
6. **Actuators / Motor Mixer:** แปลงแรงขับรวมเป็นรอบหมุนมอเตอร์ไร้แปรงถ่านทั้ง 4 ตัว

2 พจนานุกรมข้อมูลหลัก (Core Data Dictionary)

ตารางแสดงรายการข้อมูลที่ระบบต้องใช้งาน แหล่งที่มา และโมดูลที่นำไปประมวลผล:

ตัวแปร (Variable)	มิติ (Shape)	หน่วย	แหล่งเซนเซอร์	บทบาทและเหตุผลการใช้งาน (Purpose & Rationale)
pos (**p**)	float64[3]	เมตร (m)	GPS / UWB / VIO	พิกัดตำแหน่ง (x, y, z) ในพิกัดโลก เพื่อนำทางและตรวจวัดระยะห่างสิ่งกีดขวาง
vel (**v**)	float64[3]	m/s	EKF Estimator	เวกเตอร์ความเร็ว เพื่อคำนวณพลังงานจลน์และระยะเบรกหยุดนิ่ง (Deadbeat Braking)
rot (R)	float64[3,3]	ไร้หน่วย	IMU / AHRS	เมทริกซ์การวางตัว $SO(3)$ ชี้ทิศทางเวกเตอร์แรงยกของมอเตอร์
omega (ω)	float64[3]	rad/s	ไจโรสโคป (IMU)	ความเร็วเชิงมุม เพื่อคุมเสถียรภาพไม่ให้โดรนหมุนควงสว่าน
tilt_angle (θ)	float64	เรเดียน	คำนวณจาก R	ควบคุมมุมเอียงไม่ให้เกิน 35° เพื่อป้องกันการสูญเสียแรงยกแนวดิ่ง
sigma_p, sigma_v	float64	$m, m/s$	EKF Covariance	ค่าความไม่แน่นอนของเซนเซอร์ ใช้ขยายระยะขอบเขตความปลอดภัย (Robust Tube)
martingale (M_t)	float64	≥ 0	Ville e-Process	ตรวจจับการถูกสปูฟพิกัด (Sensor Spoofing) ถ้าเกินเกณฑ์จะสั่งเบรกฉุกเฉินทันที
obs_pos ($\mathbf{c}_{\text{obs}}$)	float64[3]	เมตร (m)	Depth / LiDAR	จุดกึ่งกลางสิ่งกีดขวาง เพื่อคำนวณฟังก์ชันบาเรีย $h(\mathbf{p}) \geq 0$
obs_radius (r_{obs})	float64	เมตร (m)	Object Tracker	รัศมีพื้นที่ปลอดภัยของสิ่งกีดขวาง
obs_vel ($\mathbf{v}_{\text{obs}}$)	float64[3]	m/s	Tracking Filter	ความเร็วสิ่งกีดขวาง เพื่อหลบหลีกวัตถุที่กำลังพุ่งสวนมาแบบไดนามิก
corridor_planes	list[(n, d)]	$\mathbf{n} \in \mathbb{R}^3, d \in \mathbb{R}$	Convex Decomp	ระนาบระบุขอบเขตช่องทางบิน $\mathbf{n}^\top \mathbf{p} \leq d$ เพื่อลอดช่องแคบ/ประตู
battery_soc	float64	$[0.0, 1.0]$	Smart BMS	พลังงานคงเหลือในแบตเตอรี่ ป้องกันการตกเพราะแรงดันตกฮวบ (Voltage Sag)

3 เจาะลึกเหตุผลทางฟิสิกส์และความปลอดภัย (Physics & Safety Rationale)

3.1 ทำไมต้องมีตำแหน่ง ความเร็ว และมุมเอียง? (6-DOF Underactuated Dynamics)

โดรน 4 ใบพัดเป็นระบบที่มีตัวกระทำแรงน้อยกว่าองศาความอิสระ (Underactuated System):

- โดรนไม่มีใบพัดที่ดันไปทางซ้ายหรือขวาโดยตรง การจะบินไปข้างหน้าหรือเลี้ยว จำเป็นต้อง **"เอียงตัว" (Tilt)** เพื่อแบ่งสัดส่วนแรงขับมอเตอร์
- หากไม่มีการตรวจวัด ω (Angular Velocity) และ R โดรนจะไม่สามารถทรงตัวได้และคว่ำตกทันที
- ในระบบ NMPC-CBF เรากำหนดข้อจำกัด **Tilt Cone Constraint**:

$$\sqrt{F_x^2 + F_y^2} \leq F_z \tan(\theta_{\max}), \quad \theta_{\max} = 35^\circ \tag{1}$$

  เพื่อรับประกันว่าโดรนจะมีแรงยก F_z เพียงพอต่อสู้กับแรงโน้มถ่วง mg เสมอ

3.2 ทำไมต้องใช้ความแปรปรวน (σ) และ Martingale Wealth (M_t)? (Epistemic Safety)

ในภารกิจจริง เซนเซอร์มีโอกาสผิดพลาด หลุด หรือถูกสัญญาณลวง (GPS Spoofing):

1. **ขยายเกราะคุ้มกันตามความจริง (Robust Tube Expansion):** หากเซนเซอร์มีความคลาดเคลื่อน σ_p สูง เกราะความปลอดภัยของโดรนจะถูกขยายรัศมีออกไปอัตโนมัติด้วยอัตรา $r_{\text{safe}} = r_{\text{drone}} + r_{\text{obs}} + \kappa_\delta \sigma_p$
2. **การตรวจจับด้วยทฤษฎีมาร์ติงเกลของ Ville (Ville Supermartingale Invariant):**

$$\mathbb{P}_{\mathcal{H}_0}\left(\exists t \geq 1 : M_t \geq \frac{1}{\alpha}\right) \leq \alpha \tag{2}$$

   หากสัญญาณเซนเซอร์เกิดความคลาดเคลื่อนผิดรูปอย่างต่อเนื่อง ค่า M_t จะทวีคูณสูงขึ้นจนทะลุ $1/\alpha$ โดรนจะตัดเข้าสู่สถานะ **Observability Abstain** เพื่อสั่งเบรกหยุดนิ่งกลางอากาศทันที (0.0% False Arrival)

3.3 ทำไมต้องมี Corridor Polytopes แทนที่จะใช้ทรงกลมอย่างเดียว?

เมื่อโดรนต้องบินผ่านประตูห้อง หน้าต่าง หรือช่องแคบระหว่างกำแพง:

- หากมองสิ่งกีดขวางเป็น **"ก้อนทรงกลม" (Sphere)** รัศมีของทรงกลมจะบวมทับกันจนปิดช่องประตู ทำให้โดรนประเมินว่าช่องทางนั้นตัน
- ระบบจึงแปลงข้อมูลกลุ่มจุด (Point Cloud) เป็น **3D Euclidean Signed Distance Field (ESDF)** และขยายเป็น **Convex Corridor Polytopes** กำหนดเป็นชุดอสมการเชิงระนาบ:

$$\mathcal{P} = \{\mathbf{x} \in \mathbb{R}^3 : \mathbf{a}_k^\top \mathbf{x} \leq b_k, \quad k = 1, \ldots, K\} \tag{3}$$

  ช่วยให้โดรนสามารถบินผ่านช่องประตูได้อย่างแม่นยำและปลอดภัย 100%

3.4 ทำไมข้อมูลที่ป้อนเข้าสมอง AI (Safe RL) ต้องมี 20 มิติพอดี?

โมเดล Actor-Critic ได้รับการปรับแต่งให้ใช้ชุดข้อมูลแบบ **Minimal Sufficient State (ข้อมูลน้อยที่สุดที่เพียงพอทางกายภาพ)**:

- **Error ตำแหน่ง (3 มิติ):** ชี้เป้าหมายที่ต้องบินไป
- **ความเร็ว (3 มิติ) + ความเร็วเชิงมุม (3 มิติ):** คำนวณความเฉื่อยและแรงหน่วง

- **เวกเตอร์แกน Z ของลำตัว (3 มิติ) + มุมเอียงรวม (1 มิติ):** ท่าทางการบิน
- **เวกเตอร์ระยะสิ่งกีดขวาง (3 มิติ) + ความเร็วสัมพัทธ์ (3 มิติ):** ระยะเวลาเตรียมตัวก่อนชน
- **ระดับแบตเตอรี่ (1 มิติ):** บริหารจัดการอัตราการใช้พลังงาน

หากข้อมูลน้อยกว่านี้ AI จะขาดข้อมูลเชิงฟิสิกส์ (Partial Observability) แต่หากข้อมูลเยอะกว่านี้ AI จะเรียนรู้ได้ช้ามาก

4 โค้ดโครงสร้างข้อมูลมาตรฐาน (Python Data Schema)

ทีมงานสามารถนำโค้ดภาษา Python ต่อไปนี้ไปอิมพอร์ตเพื่อส่งผ่านข้อมูลระหว่างโมดูลได้โดยตรง:

```python
from dataclasses import dataclass, field
import numpy as np

@dataclass
class DroneKinematicState:
    timestamp_s: float
    position: np.ndarray             # [x, y, z] in world frame (m)
    velocity: np.ndarray             # [vx, vy, vz] in world frame (m/s)
    rotation_matrix: np.ndarray      # 3x3 SO(3) Attitude Rotation Matrix
    angular_velocity: np.ndarray     # [wx, wy, wz] body angular rates (rad/s)
    tilt_angle_rad: float            # Body tilt angle relative to vertical
    sigma_pos: float = 0.05          # Position uncertainty std (m)
    sigma_vel: float = 0.08          # Velocity uncertainty std (m/s)
    battery_soc: float = 1.0         # Battery State of Charge [0.0, 1.0]

@dataclass
class ObstacleTarget:
    obstacle_id: int
    center: np.ndarray               # [x, y, z] center position (m)
    radius: float                    # Clearance radius (m)
    velocity: np.ndarray = field(default_factory=lambda: np.zeros(3))

@dataclass
class CorridorPlane:
    normal: np.ndarray               # Outward unit normal [nx, ny, nz]
    distance: float                  # Plane offset (normal . p <= distance)

@dataclass
class SwarmAgentPacket:
    agent_id: int
    step: int
    pos_estimate: np.ndarray         # Measured position
    sigma: float                     # Sensor reported uncertainty
    martingale_wealth: float         # Ville Martingale wealth
    valid: bool = True
```

5 การจัดโครงสร้างใน C++20 Microkernel (Zero-Allocation)

สำหรับเคอร์เนลคำนวณแรงขับความเร็วสูง (`libaegis_kernel.dylib`) ที่ทำงานระดับ < 0.1 ms ข้อมูลจะถูกจัดวางเป็น C-struct เพื่อให้ส่งผ่านหน่วยความจำได้โดยไม่ต้องคัดลอก (Zero Heap Allocation):

```cpp
struct AegisCState {
    double px, py, pz;    // World frame position [m]
    double vx, vy, vz;    // Linear velocity [m/s]
};

struct AegisCObstacle {
    double cx, cy, cz;    // Obstacle center [m]
    double radius;        // Geometric radius [m]
    double vx, vy, vz;    // Linear velocity [m/s]
};

struct AegisNmpcCorridorPlane {
    double nx, ny, nz;    // Outward hyperplane normal
    double d;             // Plane scalar bound: nx*x + ny*y + nz*z <= d
};
```
